



Handling errors without exceptions

We noted briefly in chapter 1 that throwing exceptions is a side effect. If exceptions aren't used in functional code, what *is* used instead? In this chapter, we'll learn the basic principles for raising and handling errors functionally. The big idea is that we can represent failures and exceptions with ordinary values, and we can write higher-order functions that abstract out common patterns of error handling and recovery. The functional solution, of returning errors as values, is safer and retains referential transparency, and through the use of higher-order functions, we can preserve the primary benefit of exceptions—*consolidation of error-handling logic*. We'll see how this works over the course of this chapter, after we take a closer look at exceptions and discuss some of their problems.

For the same reason that we created our own `List` data type in the previous chapter, we'll re-create in this chapter two Scala standard library types: `Option` and `Either`. The purpose is to enhance your understanding of how these types can be used for handling errors. After completing this chapter, you should feel free to use the Scala standard library version of `Option` and `Either` (though you'll notice that the standard library versions of both types are missing some of the useful functions we define in this chapter).

4.1 The good and bad aspects of exceptions

Why do exceptions break referential transparency, and why is that a problem? Let's look at a simple example. We'll define a function that throws an exception and call it.

Listing 4.1 Throwing and catching an exception

```
def failingFn(i: Int): Int = {
  val y: Int = throw new Exception("fail!")
  try {
    val x = 42 + 5
    x + y
  }
  catch { case e: Exception => 43 }
}
```

val y: Int = ... declares y as having type Int and sets it equal to the right-hand side of =.

A catch block is just a pattern-matching block like the ones we've seen. case e: Exception is a pattern that matches any Exception, and it binds this value to the identifier e. The match returns the value 43.

Calling `failingFn` from the REPL gives the expected error:

```
scala> failingFn(12)
java.lang.Exception: fail!
  at .failingFn(<console>:8)
  ...
```

We can prove that `y` is not referentially transparent. Recall that any RT expression may be substituted with the value it refers to, and this substitution should preserve program meaning. If we substitute `throw new Exception("fail!")` for `y` in `x + y`, it produces a different result, because the exception will now be raised inside a `try` block that will catch the exception and return 43:

```
def failingFn2(i: Int): Int = {
  try {
    val x = 42 + 5
    x + ((throw new Exception("fail!")): Int)
  }
  catch { case e: Exception => 43 }
}
```

A thrown Exception can be given any type; here we're annotating it with the type Int.

We can demonstrate this in the REPL:

```
scala> failingFn2(12)
res1: Int = 43
```

Another way of understanding RT is that the meaning of RT expressions *does not depend on context* and may be reasoned about locally, whereas the meaning of non-RT expressions is *context-dependent* and requires more global reasoning. For instance, the meaning of the RT expression `42 + 5` doesn't depend on the larger expression it's embedded in—it's always and forever equal to 47. But the meaning of the expression `throw new Exception("fail")` is very context-dependent—as we just demonstrated, it takes on different meanings depending on which `try` block (if any) it's nested within.

There are two main problems with exceptions:

- As we just discussed, *exceptions break RT and introduce context dependence*, moving us away from the simple reasoning of the substitution model and making it possible to write confusing exception-based code. This is the source of the folklore advice that exceptions should be used only for error handling, not for control flow.